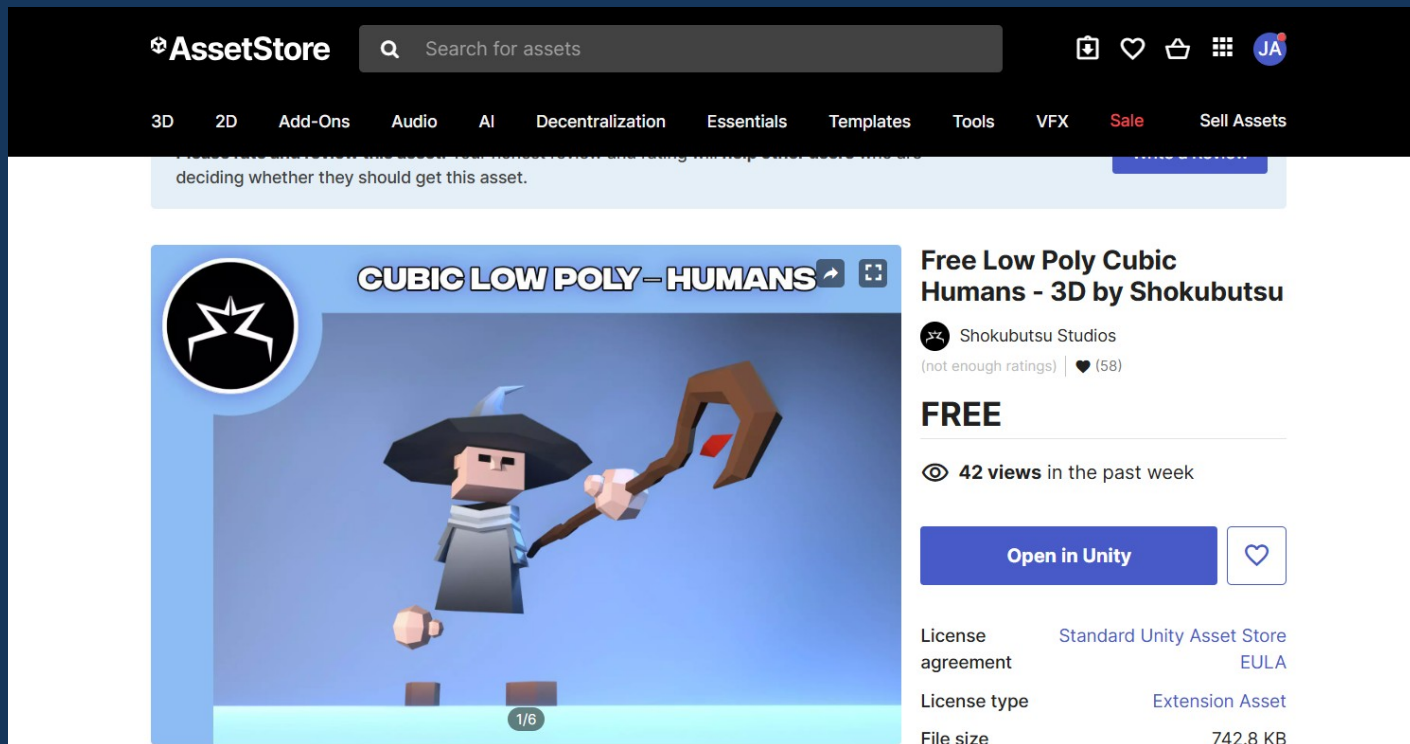


Esportador de formato GLTF desde Unity

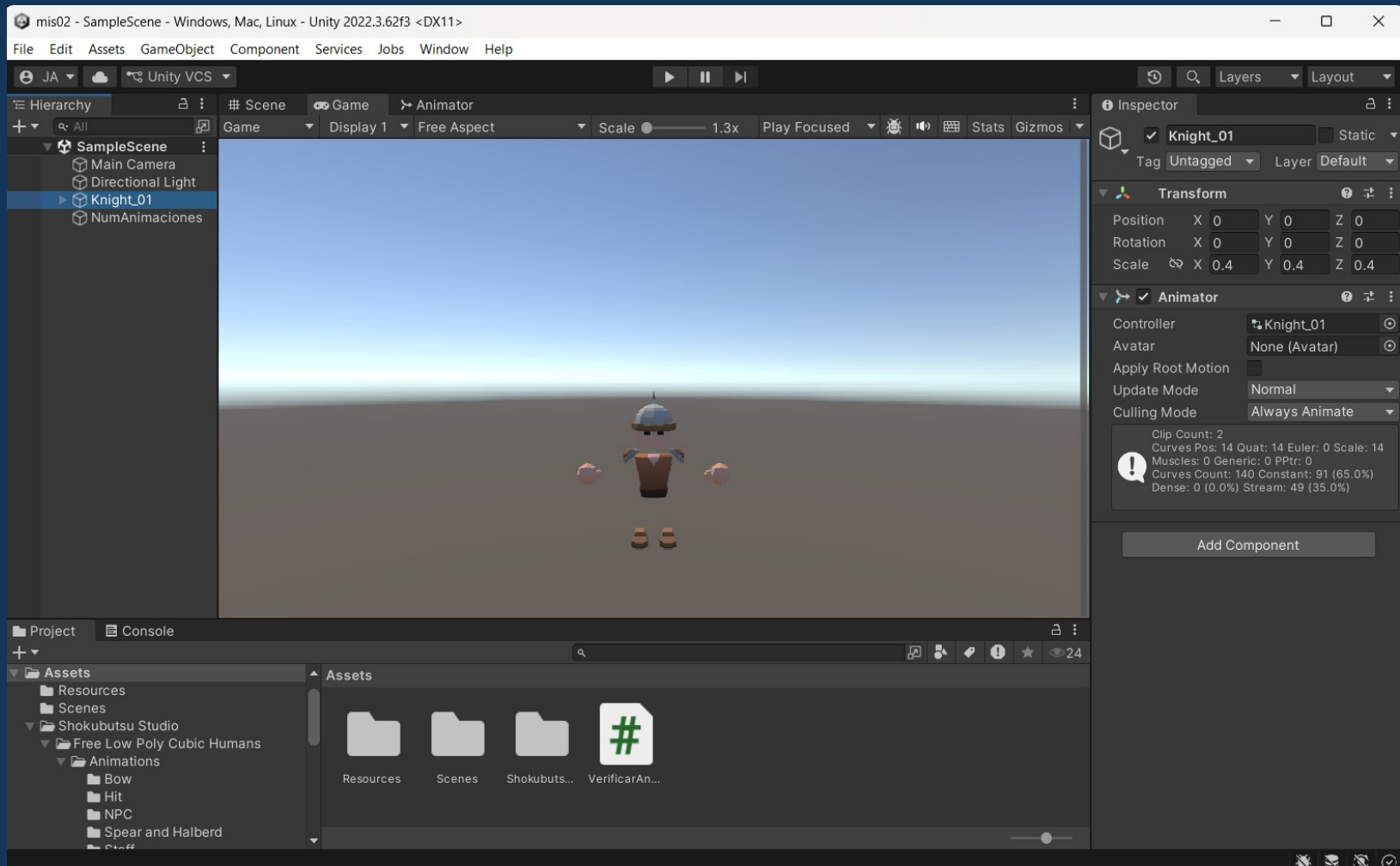
Asset Utilizado

- Personaje utilizado como prueba. Este asset es utilizado como material educativo sin fines de lucro. Los alumnos deberán generar sus propios modelos



Asset en Unity

- Personaje utilizado como prueba. Este asset es utilizado como material educativo sin fines de lucro. Los alumnos deberán generar sus propios modelos



Guía Técnica: Exportación de Modelos Animados desde Unity a WebXR (Three.js)

- 1. Descarga e Instalación del Exportador en UnityEl paquete oficial de Khronos Group permite procesar las jerarquías de animación y empaquetarlas de forma binaria.
- Pasos para la instalación local:
 - 1. Ingrese al repositorio oficial del exportador:
<https://github.com/KhronosGroup/UnityGLTF/releases>
 - 2. Descargue el archivo comprimido .zip de la versión estable más reciente.
 - 3. En su computadora, haga clic derecho sobre el archivo descargado y seleccione Extraer todo o descomprimir.
 - 4. Abra su proyecto en Unity y diríjase a la barra de menús superior: Window → Package Manager.
 - 5. Haga clic en el botón + situado en la esquina superior izquierda de la ventana del Package Manager.
- 6. Seleccione la opción Add package from disk....
- 7. Navegue hasta la carpeta extraída, abra la subcarpeta llamada UnityGLTF, seleccione el archivo package.json y haga clic en Abrir.
- 8. Tras finalizar la compilación de scripts, Unity integrará el módulo de exportación dentro de la interfaz

Guía Técnica: Exportación de Modelos Animados desde Unity a WebXR (Three.js)

3. Control de Calidad en Entornos de Simulación Web
Antes de escribir código de control, valide el estado estructural del archivo exportado.

1. Ingrese a la plataforma de pruebas web: Babylon.js Sandbox.
2. Arrastre el archivo generado Knight_01.glb directamente al centro de la ventana del navegador.
3. Despliegue el menú inferior derecho (icono de reproducción/claqueta).
4. Verifique que el listado indexe por separado cada animación:

Índice [0]: NPC_Walk

Índice [1]: NPC_Run

4. Implementación en Three.js con Soporte WebXR

- 4. Implementación en Three.js con Soporte WebXR El cargador nativo GLTFLoader permite decodificar el archivo sin desfases en los factores de escala (estableciendo el valor real en 1 para el asset optimizado).

```
// Load GLTF Model and Animation (Knight_01)
const loader = new GLTFLoader();
loader.load( 'models/gltf/Knight_01.glb', function ( gltf ) {
```

mis02 - Github

XRUS1/mis02

github.com/XRUS1/mis02

XRUS1 / mis02

Type / to search

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

mis02 Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Code

XRUS1 Refactor GLTF model loading comments and scaling ✓ fd24f3d · yesterday 23 Commits

build	Add files via upload	yesterday
jsn	Add files via upload	yesterday
models	Add files via upload	yesterday
textures	Add files via upload	yesterday
README.md	Correct capitalization in README title	yesterday
files.json	Add files via upload	yesterday
index.html	Refactor GLTF model loading comments and scaling	yesterday
main.css	Add files via upload	yesterday
tags.json	Add files via upload	yesterday

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Deployments 8

github-pages yesterday

index.html – Cargador GLTF

```
69      // Load GLTF Model and Animation (Knight_01)
70      const loader = new GLTFLoader();
71      loader.load( 'models/gltf/Knight_01.glb', function ( gltf ) {
72          const object = gltf.scene;
73
74          // CORRECCIÓN DEFINITIVA DE ESCALA: Al ser .glb corregido de Unity, se usa escala real 1
75          object.scale.set(1, 1, 1);
76          object.position.x = 0.8;
77          object.position.y = -0.5;
78          object.position.z = -1.5;
79          // object.rotation.y = Math.PI / 2; // Rotando 90
80
81          mixer = new THREE.AnimationMixer( object );
82          const action = mixer.clipAction( gltf.animations[ 0 ] );
83          action.play();
84
85          object.traverse( function ( child ) {
86              if ( child.isMesh ) {
87                  child.castShadow = true;
88                  child.receiveShadow = true;
89              }
90          } );
91
92          scene.add( object );
93      } );
94
```


Referencias Bibliográficas